PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

BRUNNO ALEXCSON DE CARVALHO

TRABALHO 1 - ORDENAÇÃO E BUSCA

Relatório apresentado como requisito parcial para obtenção de nota na disciplina de Estrutura de Dados II.

Docente: Thiago Augusto

Goiânia – GO
2026

Sumário

1 Introdução **2**

2 Objetivos **3**
2.1 Objetivo Geral . . . 3
2.2 Objetivos Específicos . . . 3

3 Algoritmos Implementados **4**
3.1 Algoritmos de Ordenação . . . 4
3.1.1 Bubble Sort . . . 4
3.1.2 Selection Sort . . . 5
3.1.3 Merge Sort . . . 6
3.2 Algoritmos de Busca . . . 7
3.2.1 Busca Sequencial . . . 7
3.2.2 Busca Binária . . . 7

4 Metodologia **8**
4.1 Tamanhos de Entrada . . . 8
4.2 Tipos de Dados . . . 8
4.3 Métricas de Avaliação . . . 8
4.4 Reprodutibilidade dos Testes . . . 8
4.5 Procedimento Experimental . . . 8

5 Resultados Experimentais **9**
5.1 Resultados dos Algoritmos de Ordenação . . . 9
5.2 Resultados dos Algoritmos de Busca . . . 9

6 Análise dos Resultados **10**
6.1 Algoritmos de Ordenação . . . 10
6.2 Algoritmos de Busca . . . 10
6.3 Considerações Gerais . . . 10

7 Conclusão **11**

1 Introdução

Este trabalho tem como objetivo implementar e analisar algoritmos clássicos de ordenação e busca, com foco em seu comportamento prático e desempenho em diferentes cenários.

Foram desenvolvidos diferentes algoritmos de ordenação e busca, juntamente com uma aplicação que permite a execução de testes experimentais variando o tamanho e a organização dos dados de entrada.

A proposta central é comparar o desempenho dos algoritmos, utilizando métricas como tempo de execução, número de comparações e trocas realizadas, relacionando os resultados obtidos com a teoria de complexidade dos algoritmos.

2 Objetivos

2.1 Objetivo Geral

Implementar e analisar algoritmos de ordenação e busca, avaliando seu desempenho em diferentes cenários.

2.2 Objetivos Específicos

- Implementar algoritmos de ordenação;
- Implementar busca sequencial e busca binária;
- Realizar testes experimentais com diferentes volumes de dados;
- Comparar o desempenho dos algoritmos;
- Relacionar resultados práticos com a teoria.

3 Algoritmos Implementados

3.1 Algoritmos de Ordenação

3.1.1 Bubble Sort

O Bubble Sort é um algoritmo simples que percorre o vetor repetidamente, trocando elementos adjacentes quando estão fora de ordem.

No melhor caso (vetor já ordenado), possui complexidade $O(n)$. No pior caso, possui complexidade $O(n^2)$. No caso médio, também apresenta complexidade $O(n^2)$.

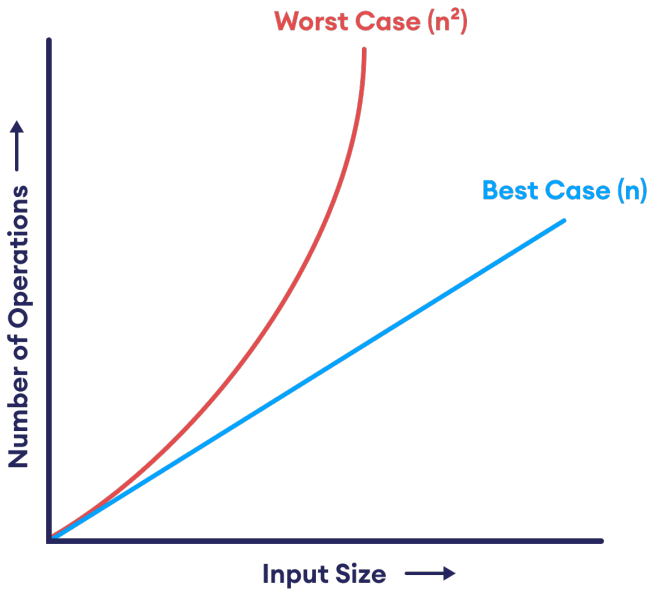

Figura 1: Gráfico de Complexidade do Bubble Sort

Como observado na Figura 1, o algoritmo Bubble Sort apresenta comportamentos distintos dependendo da organização inicial dos dados.

No melhor caso, quando o vetor já está ordenado, o algoritmo realiza apenas uma passagem pelo vetor, resultando em complexidade $O(n)$. Isso ocorre quando há uma verificação de troca que permite encerrar o algoritmo antecipadamente.

Por outro lado, no pior caso, quando o vetor está em ordem decrescente, o algoritmo realiza o número máximo de comparações e trocas, resultando em complexidade $O(n^2)$. Nesse cenário, o crescimento do tempo de execução é claramente quadrático, como evidenciado pelo gráfico.

Dessa forma, o Bubble Sort é altamente sensível à organização inicial dos dados, apresentando bom desempenho apenas em casos específicos.

3.1.2 Selection Sort

O Selection Sort seleciona o menor elemento do vetor e o coloca na posição correta a cada iteração. Sua complexidade é sempre $O(n^2)$, independentemente da ordem inicial dos dados.

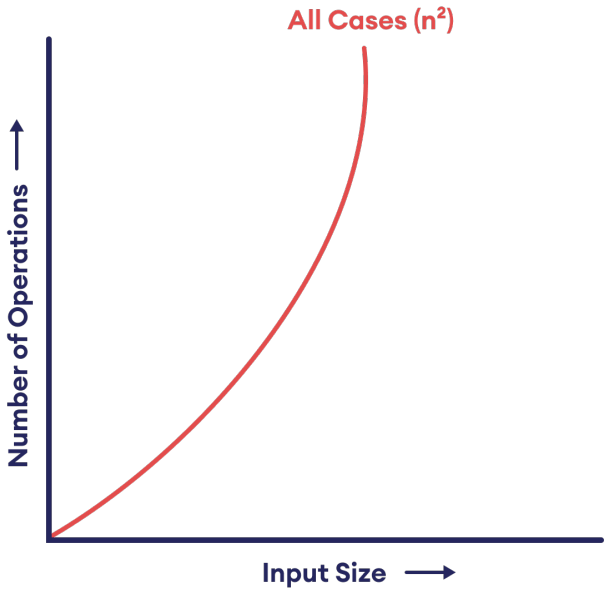

Figura 2: Gráfico de Complexidade do Selection Sort

Conforme apresentado na Figura 2, o algoritmo Selection Sort mantém um comportamento consistente independentemente da ordem inicial dos dados.

Diferentemente do Bubble Sort, o Selection Sort sempre percorre todo o vetor em busca do menor elemento a cada iteração, realizando um número fixo de comparações. Por esse motivo, sua complexidade permanece $O(n^2)$ tanto no melhor quanto no pior caso.

O gráfico evidencia esse comportamento uniforme, com crescimento quadrático constante, sem variações significativas entre diferentes tipos de entrada.

Embora apresente simplicidade de implementação, o Selection Sort não é eficiente para grandes volumes de dados, sendo superado por algoritmos mais otimizados, como o Merge Sort.

3.1.3 Merge Sort

O Merge Sort é um algoritmo baseado na estratégia de divisão e conquista, que divide o vetor recursivamente e depois realiza a intercalação dos elementos.

Possui complexidade $O(n \log n)$ em todos os casos, sendo mais eficiente para grandes volumes de dados.

Detalhe importante: não há contagem direta de trocas, pois o algoritmo trabalha com cópias durante o processo de intercalação.

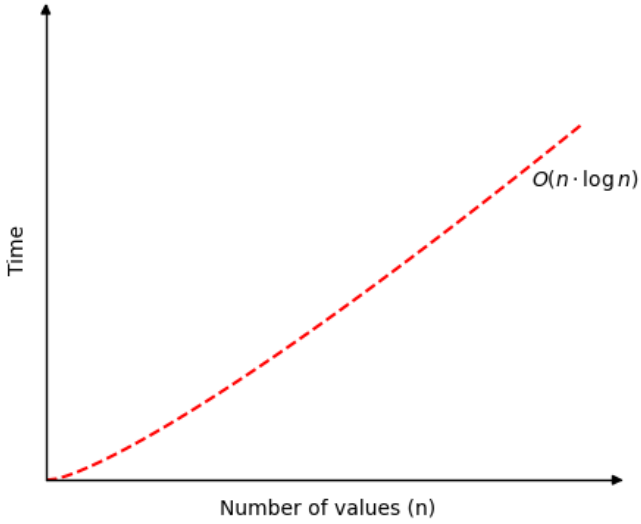

Figura 3: Gráfico de Complexidade do Merge Sort

Conforme ilustrado na Figura 3, o algoritmo Merge Sort apresenta comportamento consistente em todos os cenários analisados.

Diferentemente dos algoritmos quadráticos, o Merge Sort utiliza a estratégia de divisão e conquista, dividindo recursivamente o vetor em partes menores até atingir subvetores de tamanho unitário, e posteriormente realizando a intercalação ordenada desses elementos.

Essa abordagem resulta em complexidade $O(n \log n)$ tanto no melhor, quanto no pior e no caso médio, uma vez que o número de divisões do vetor cresce de forma logarítmica, enquanto o processo de intercalação percorre todos os elementos a cada nível da recursão.

O gráfico evidencia esse crescimento mais eficiente quando comparado aos algoritmos $O(n^2)$, especialmente para grandes volumes de dados, onde a diferença de desempenho se torna significativamente mais perceptível.

Portanto, o Merge Sort se destaca como uma solução mais adequada para conjuntos de dados maiores, apesar de demandar maior uso de memória auxiliar devido às operações de cópia durante a intercalação.

3.2 Algoritmos de Busca

3.2.1 Busca Sequencial

A busca sequencial percorre o vetor elemento por elemento até encontrar o valor desejado ou atingir o final da estrutura.

No melhor caso, quando o elemento está na primeira posição, a complexidade é $O(1)$. No pior caso, quando o elemento está na última posição ou não está presente, a complexidade é $O(n)$.

3.2.2 Busca Binária

A busca binária é aplicada sobre vetores previamente ordenados, dividindo repetidamente o espaço de busca pela metade.

No melhor caso, quando o elemento encontra-se na posição central já na primeira verificação, a complexidade é $O(1)$.

Essa estratégia reduz significativamente o número de comparações necessárias, resultando em complexidade $O(\log n)$.

Para a utilização desse algoritmo, o vetor foi previamente ordenado utilizando o algoritmo Merge Sort.

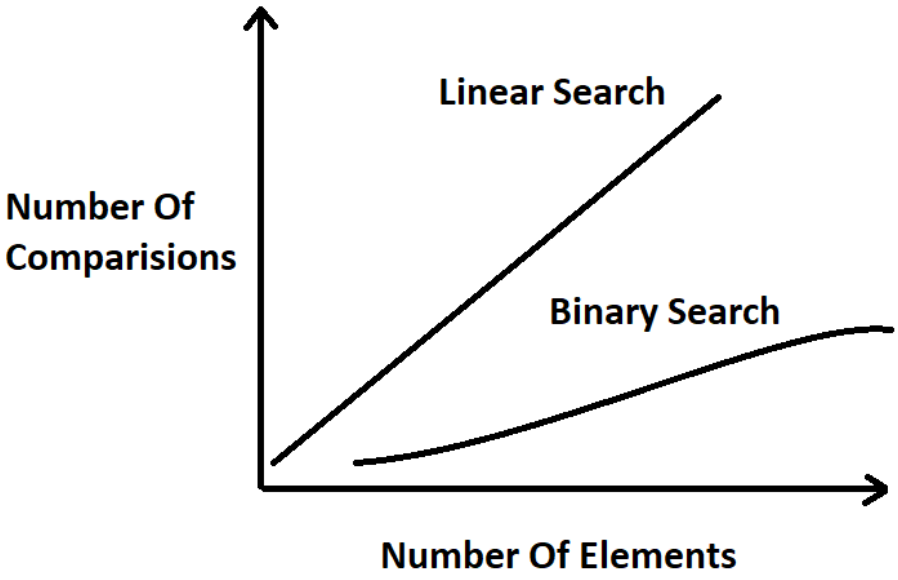

Figura 4: Gráfico de Comparação de Busca Sequencial e Busca Binária

4 Metodologia

Os testes experimentais foram realizados com o objetivo de avaliar o desempenho dos algoritmos de ordenação e busca em diferentes cenários.

4.1 Tamanhos de Entrada

Foram utilizados diferentes tamanhos de entrada para análise de desempenho, classificados como:

- Pequeno: 100 elementos
- Médio: 1.000 elementos
- Grande: 10.000 elementos
- Variável: tamanho definido pelo usuário em tempo de execução

Essa variação permite observar o comportamento dos algoritmos à medida que o volume de dados aumenta.

4.2 Tipos de Dados

Além do tamanho, também foram considerados diferentes tipos de organização dos dados:

- Ordenado crescente
- Ordenado decrescente
- Aleatório

Essa abordagem permite analisar o impacto da ordem inicial dos dados no desempenho dos algoritmos, especialmente em algoritmos sensíveis como o Bubble Sort.

4.3 Métricas de Avaliação

Para a análise comparativa, foram utilizadas as seguintes métricas:

- Tempo de execução (em segundos)
- Número de comparações realizadas
- Número de trocas realizadas (quando aplicável)

O tempo de execução foi medido durante a execução dos algoritmos, permitindo avaliar sua eficiência prática. Já as métricas de comparações e trocas foram contabilizadas diretamente no código, fornecendo uma visão mais detalhada do comportamento interno de cada algoritmo.

4.4 Reprodutibilidade dos Testes

Para garantir a consistência dos resultados, foi utilizada uma semente fixa na geração de números aleatórios, por meio da função `srand(42)`.

Isso assegura que os mesmos conjuntos de dados aleatórios sejam gerados em diferentes execuções, permitindo a reprodução dos experimentos e a comparação justa entre os algoritmos.

4.5 Procedimento Experimental

Para cada algoritmo, foram realizados testes considerando todas as combinações de tamanho de entrada e organização dos dados.

No caso da busca binária, os vetores foram previamente ordenados utilizando o algoritmo Merge Sort, garantindo a validade da aplicação do método.

Os resultados obtidos foram registrados e posteriormente analisados por meio de tabelas e gráficos, permitindo a comparação direta entre os algoritmos estudados.

5 Resultados Experimentais

5.1 Resultados dos Algoritmos de Ordenação

A Tabela a seguir apresenta os resultados obtidos para os algoritmos de ordenação, considerando diferentes tamanhos de entrada. Os resultados apresentados foram obtidos a partir de vetores com valores gerados aleatoriamente, representando o comportamento médio dos algoritmos.

Tempo de Execução (segundos)

Algoritmo	**n = 100**	**n = 1000**	**n = 10000**
Bubble Sort	0.000000s	0.003000s	0.126000s
Selection Sort	0.000000s	0.001000s	0.054000s
Merge Sort	0.000000s	0.000000s	0.000000s

Número de Comparações

Algoritmo	**n = 100**	**n = 1000**	**n = 10000**
Bubble Sort	4.814	497.670	49.986.354
Selection Sort	4.950	499.500	49.995.000
Merge Sort	550	8.723	120.447

Número de Trocas

Algoritmo	**n = 100**	**n = 1000**	**n = 10000**
Bubble Sort	2.432	255.413	24.998.808
Selection Sort	93	989	9.989
Merge Sort	0	0	0

5.2 Resultados dos Algoritmos de Busca

A Tabela a seguir apresenta a quantidade de comparações realizadas pelos algoritmos de busca. Para a análise apresentada na tabela, considera-se a busca por um elemento que não está presente no vetor.

Número de Comparações nas Buscas

Algoritmo	**n = 100**	**n = 1000**	**n = 10000**
Busca Sequencial	100	1.000	10.000
Busca Binária	≈ 7	≈ 10	≈ 14

Obs.: Os valores correspondem ao número de comparações realizadas em cada execução.

6 Análise dos Resultados

A partir dos resultados experimentais obtidos, é possível observar diferenças significativas no desempenho dos algoritmos de ordenação e busca, tanto em termos de tempo de execução quanto no número de operações realizadas.

6.1 Algoritmos de Ordenação

Analisando o tempo de execução, nota-se que os algoritmos Bubble Sort e Selection Sort apresentam crescimento significativo conforme o tamanho da entrada aumenta. Para $n = 10000$, o Bubble Sort atingiu aproximadamente 0.126 segundos, enquanto o Selection Sort apresentou desempenho ligeiramente melhor, com cerca de 0.054 segundos.

Esse comportamento é esperado, uma vez que ambos possuem complexidade quadrática $O(n^2)$, o que implica aumento expressivo no número de operações à medida que o tamanho do vetor cresce.

O Merge Sort, por sua vez, apresentou tempo de execução praticamente nulo nos testes realizados. Isso ocorre devido à alta eficiência do algoritmo, cuja complexidade é $O(n \log n)$, além de limitações na precisão da medição de tempo para valores muito pequenos. Esses valores próximos de zero indicam que o tempo de execução é muito pequeno para ser medido com precisão pelo método utilizado, não significando custo nulo.

Ao analisar o número de comparações, observa-se que o Bubble Sort e o Selection Sort apresentam valores muito próximos, atingindo aproximadamente 50 milhões de comparações para $n = 10000$, confirmando o crescimento quadrático esperado.

Já o Merge Sort apresentou um número significativamente menor de comparações (120.447 para $n = 10000$), evidenciando sua superioridade em termos de eficiência.

Em relação ao número de trocas, o Bubble Sort apresentou valores extremamente elevados, o que impacta diretamente seu desempenho. O Selection Sort, embora também quadrático, realiza um número bem menor de trocas, o que explica seu desempenho ligeiramente superior em tempo de execução.

O Merge Sort não apresentou trocas diretas, pois utiliza um processo de cópia durante a intercalação, o que caracteriza uma abordagem diferente dos algoritmos baseados em troca.

6.2 Algoritmos de Busca

Para os algoritmos de busca, foi considerado o pior caso, em que o elemento buscado não está presente no vetor.

A busca sequencial apresentou crescimento linear no número de comparações, com valores diretamente proporcionais ao tamanho da entrada (100, 1000 e 10000), confirmando sua complexidade $O(n)$.

Por outro lado, a busca binária apresentou crescimento muito mais lento, com aproximadamente 7, 10 e 14 comparações para os respectivos tamanhos de entrada. Esse comportamento está de acordo com a complexidade $O(\log n)$, característica desse algoritmo.

A diferença entre os dois métodos torna-se mais evidente à medida que o tamanho dos dados aumenta, demonstrando que a busca binária é significativamente mais eficiente para grandes volumes de dados, desde que o vetor esteja previamente ordenado.

6.3 Considerações Gerais

De modo geral, os resultados obtidos estão em conformidade com a teoria de análise de algoritmos. Algoritmos com complexidade $O(n^2)$ mostraram-se inviáveis para grandes volumes de dados, enquanto algoritmos mais eficientes, como o Merge Sort e a busca binária, apresentaram desempenho superior.

Esses resultados reforçam a importância da escolha adequada do algoritmo de acordo com o contexto e o tamanho dos dados a serem processados.

7 Conclusão

Este trabalho permitiu a implementação e análise prática de algoritmos clássicos de ordenação e busca, evidenciando suas diferenças de desempenho em distintos cenários.

A partir dos resultados obtidos, foi possível observar que algoritmos de ordenação com complexidade quadrática, como Bubble Sort e Selection Sort, apresentam desempenho limitado para grandes volumes de dados, devido ao elevado número de comparações e operações realizadas. Em contrapartida, o Merge Sort demonstrou maior eficiência, mantendo desempenho consistente mesmo para entradas maiores, em função de sua complexidade $O(n \log n)$.

No contexto de busca, verificou-se que a busca sequencial, apesar de simples, torna-se ineficiente à medida que o tamanho do vetor aumenta, enquanto a busca binária apresentou desempenho significativamente superior, realizando um número reduzido de comparações, desde que aplicada sobre dados previamente ordenados.

De modo geral, os experimentos reforçam a importância da escolha adequada de algoritmos conforme o problema e o volume de dados envolvidos. Além disso, o trabalho contribuiu para a consolidação dos conceitos teóricos por meio da aplicação prática, proporcionando uma melhor compreensão sobre eficiência e análise de algoritmos.